Bellerophon Language Documentation

Language Overview

Bellerophon is a domain-specific scripting language (DSL) for 3D printing. Write clean, procedural motions and parametric macros that compile directly to G-code for Klipper or Marlin firmware.

> **Note:**
>
> **Philosophy:** Precise, readable commands that make controlling 3D printers easier.

Getting Started

Bellerophon is a compiled Domain-Specific Language (DSL) and IDE for additive manufacturing. It allows you to write structured, parametric scripts and compile them into firmware-compatible G-code macros. This section describes the setup and workflow for the Bellerophon development environment.

1. System Requirements

- **Bundle:** Bellerophon includes a bundled JRE, eliminating the need for manual Java installation.
- **Hardware:** Any modern Windows, Linux, or macOS system.

2. Launching the IDE

The Bellerophon IDE is delivered as a self-contained environment. Simply launch the provided executable to initialize the local server.

Windows

- Navigate to the project root folder.
- Double-click `run.bat`. This will initialize the bundled JRE and start the Bellerophon server.
- The IDE will automatically launch in your default web browser at `http://localhost:4567`.
- To shut down, simply close the server terminal window.

Linux / Mac

- Navigate to the project root folder in your terminal.
- Ensure the launch script is executable: `chmod +x run.sh`
- Run the script: `./run.sh`
- The IDE will be accessible at `http://localhost:4567`.
- Press `Ctrl+C` in the terminal to terminate the server.

> **Note:**
>
> The server runs locally on port 4567. No internet connection required after installation.

3. Compilation Architecture

Bellerophon uses a decoupled **Firmware Adapter Architecture**. Your `.bph` source code remains universal, while the compiler handles the conversion to firmware-specific instructions via targeted adapters.

- **Universal Source:** Write once, deploy anywhere. Your logic is maintained in a single Bellerophon project regardless of the printer controller.
- **Modular Adapters:** Choose your target in the IDE prior to compilation. The compiler applies the appropriate syntax rules and feature-set limitations for that specific environment (e.g., Klipper, Marlin, or future custom firmware targets).
- **Firmware Independence:** The Bellerophon engine is designed to be firmware-agnostic; as new manufacturing controllers emerge, new adapters can be integrated without modifying your existing scripts.

4. IDE & Workflow

- **Input:** Bellerophon processes `.bph` scripts. The IDE provides real-time, line-oriented syntax validation and error highlighting as you develop.
- **Debugging:** Access the **Build Log** and **Exceptions** tabs for immediate feedback on compilation errors.
- **Session Management:** The IDE automatically preserves your workspace, restoring open files and settings to prevent data loss.
- **Reference Panel:** Access all Bellerophon tokens and documentation directly within the IDE sidebar to reduce syntax lookups and speed up your workflow.

5. Core Capabilities

Bellerophon is a **Parametric Engine**, moving manufacturing beyond static G-code into procedurally generated motion.

Parametric Geometry	**Smart Extrusion**

Use global variables, nested `Brepeat` loops, and advanced math (`sin`, `cos`, `sqrt`, `pi`) to define complex motion paths programmatically.

Utilize the `EnableAutoExtrude` engine for path-aware volume calculation, or perform manual overrides for high-precision extrusion control.

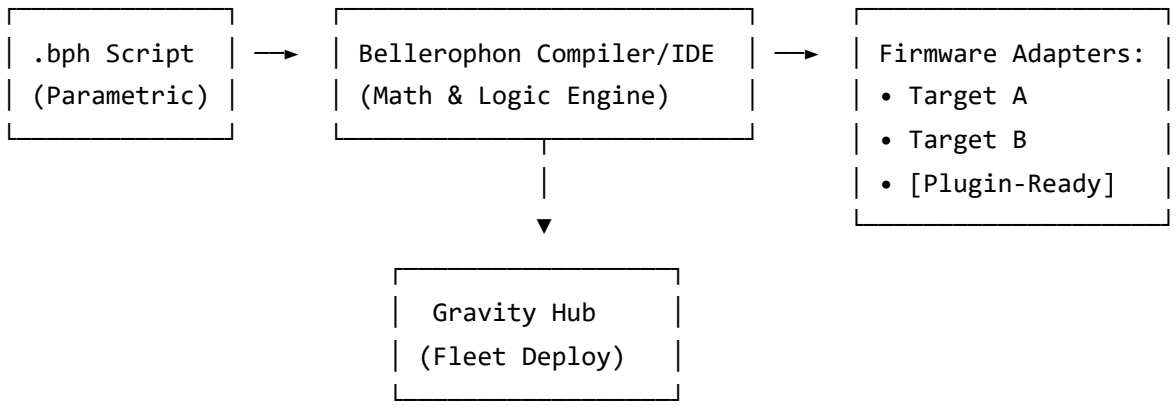
Because these features are evaluated by the **Bellerophon Compiler** at build-time, your final macro output is optimized, validated, and ready for hardware execution. This offloads the computational weight of complex mathematics from your printer's controller, ensuring fluid motion even for highly intricate, procedural geometry.

6. Technical Considerations & Limitations

- **DSL Purpose:** Bellerophon is a compilation engine, not a 3D slicer; it does not process 3D meshes (STL/STEP).
- **Variable Scope:** Variables are currently local to the macro scope to prevent collision. Global state persistence is a planned future architectural expansion.
- **Target Compatibility:** Feature support (such as conditional logic) is determined by the specific capabilities of the firmware target adapter selected at compile-time.

7. Bellerophon Architecture & Workflow

Bellerophon operates as a high-level parametric engine, abstracting complex geometry into firmware-specific instructions via a modular adapter system.



> **Note:**
>
> **Extensible Architecture:** Bellerophon uses a decoupled compiler design. New firmware targets can be added by implementing a new `FirmwareAdapter`, allowing the DSL to remain constant even as the manufacturing landscape changes.

Program Structure

A Bellerophon program consists of one or more macros. Each macro starts with `M.title` and ends with `M.end`.

```
M.title "Macro Name"
    [statement 1]
    [statement 2]
    [statement 3]
M.end
```

Each command must be on its own line. Indentation is optional but recommended. Comments start with `#`.

Local User Variables

Bellerophon supports **local variables** defined inside macros. Variables are scoped to the macro they are defined in and are not accessible outside it.

> **Note:**
>
> **Scope:** Variables are local to the macro. They do not persist across macro calls. Global variables are a planned future feature.

Assignment Syntax

To define or update a variable, use the assignment statement:

```
variable_name = expression
```

Supported Expressions

Variables can be assigned any valid expression:

- **Numbers:** `150`, `3.14`, `-5`
- **Arithmetic:** +, -, *, /, ^ (power)
- **Parentheses:** `(base + offset) * 2`
- **Functions:** `sin()`, `cos()`, `tan()`, `sqrt()`, `abs()`, `sign()`

- **Constant:** `pi`
- **Variable references:** `my_x`, `speed` (must be defined earlier in the macro)

Example

```
M.title "example"

    base = 100

    offset = 50

    speed = (base + offset) * 2

    SetSpeed = speed

    MoveTo x=base y=offset

    MoveTo x=base+offset y=base-offset

M.end
```

Output

```
[gcode_macro example]

gcode:

  G1 F300

  G1 X100.000 Y50.000

  G1 X150.000 Y50.000
```

Statement Reference

COMMAND	SYNTAX	EXAMPLE	G-CODE / NOTES
Assignment	*variable* = expression	*my_x* = 150	Stores the expression result in a local variable. Variables are scoped to the macro.
Home	Home or Home -> coordList	Home or Home -> x=0 y=0 z=0	G28 / G28 X Y Z

COMMAND	SYNTAX	EXAMPLE	G-CODE / NOTES
`Move`	`Move` **direction**	`Move` **left** / **right** / **center** / **up** / **down**	G1 with directional offset. Iterates 1 mm. **center** defaults to X0 Y0
`MoveTo`	`MoveTo` **coordList**	`MoveTo` **x=100 y=100 z=0.2**	G1 X Y Z E
`Absolute`	`Absolute`	`Absolute`	G90
`Relative`	`Relative`	`Relative`	G91
`Heat`	`Heat` **target=number**	`Heat` **extruder=200**	Sets temperature and **waits until it is reached**. Extruder → M109, Bed → M190, Chamber → M141
`Set_Heater_Temperature`	`Set_Heater_Temperature` **target=number**	`Set_Heater_Temperature` **bed=60**	Sets temperature but **does NOT wait**. Extruder → M104, Bed → M140, Chamber → M141
`WaitForTemp`	`WaitForTemp` **target**	`WaitForTemp` **extruder**	Waits until the target reaches its current set temperature. Does **not change** the temperature. Extruder → M109, Bed → M190, Chamber → M141
`Cooldown`	`Cooldown` **target** (optional)	`Cooldown` **extruder** or just `Cooldown`	M104 S0 / M140 S0 / TURN_OFF_HEATERS. If target omitted, turns off all heaters.
`TimeoutSet`	`TIMEOUT_SET` **seconds**	`TIMEOUT_SET` **300**	Set idle timeout (seconds). Maps to Klipper's `SET_IDLE_TIMEOUT TIMEOUT=300` (ignored in Marlin).
`Pause`	`Pause`	`Pause`	PAUSE (Klipper) / M0 (Marlin)
`Resume`	`Resume`	`Resume`	RESUME (Klipper) / M108 (Marlin)
`Respond`	`Respond` **MSG "text"**	`Respond` **MSG "Print complete"**	RESPOND MSG="text" (Klipper) / M117 (Marlin)
`SetSpeed`	`SetSpeed` **= number**	`SetSpeed` **= 100**	G1 F#

COMMAND	SYNTAX	EXAMPLE	G-CODE / NOTES
`SetFan`	`SetFan` **= number**	`SetFan` **= 255**	SET_FAN SPEED=# (Klipper) / M106 S# (Marlin)
`SetNozzle`	`SetNozzle` **= number**	`SetNozzle` **= 0.6**	Sets the nozzle diameter (mm) for extrusion calculations. Modifies `PrinterSettings` for the current macro. Default is taken from the printer profile UI.
`SetFilament`	`SetFilament` **= number**	`SetFilament` **= 2.85**	Sets the filament diameter (mm) for extrusion calculations. Modifies `PrinterSettings` for the current macro. Default is taken from the printer profile UI.
`SetLayerHeight`	`SetLayerHeight` **= number**	`SetLayerHeight` **= 0.3**	Sets the layer height (mm) for extrusion calculations and the `Layer` command. Modifies `PrinterSettings` for the current macro. Default is taken from the printer profile UI.
`SetExtrusionMultiplier`	`SetExtrusionMultiplier` **= number**	`SetExtrusionMultiplier` **= 0.95**	Adjusts the extrusion multiplier (flow rate compensation). Modifies `PrinterSettings` for the current macro. Default is taken from the printer profile UI.
`EnableAutoExtrude`	`EnableAutoExtrude` **= 1|0**	`EnableAutoExtrude` **= 1**	Enables (`1`) or disables (`0`) automatic extrusion calculation for `MoveTo` commands. Manual `e=...` overrides auto-extrusion.
`Layer`	`Layer` **number** ... **end**	`Layer` **5** ... **end**	Repeats the enclosed block for the specified number of layers, automatically raising Z by the current `LayerHeight` after each layer.
`PrintFile`	`PRINTFILE` **"file.gcode"**	`PRINTFILE` **"cube.gcode"**	SDCARD_PRINT_FILE FILENAME="file.gcode"

COMMAND	SYNTAX	EXAMPLE	G-CODE / NOTES
			(Klipper) / M23 (Marlin)
`Call Macro`	`M.call` **"MacroName"**	`M.call` **"CalibrateBed"**	Calls another macro (Klipper) / M98 (Marlin)
`If / EndIf`	`if` condition ... `endif`	`if` extruder > **190** ... `endif`	**Klipper only**; compiled to Jinja condition. Not available in Marlin target.
`RelativeExtrusion`	`RelativeExtrusion`	`RelativeExtrusion`	M83
`ResetExtruder`	`ResetExtruder`	`ResetExtruder`	G92 E0
`BedMeshCalibrate`	`BedMeshCalibrate`	`BedMeshCalibrate`	BED_MESH_CALIBRATE (Klipper) / G29 (Marlin)
`Level`	`Level`	`Level`	Alias for BedMeshCalibrate → triggers BED_MESH_CALIBRATE or G29
`ProbeCalibrate`	`ProbeCalibrate`	`ProbeCalibrate`	PROBE_CALIBRATE (Klipper) – not available in Marlin
`LoadBedMesh`	`LoadBedMesh` **"profile"**	`LoadBedMesh` **"default"**	BED_MESH_PROFILE LOAD=profile (Klipper) – not available in Marlin
`SetPressureAdvance`	`SetPressureAdvance` **number**	`SetPressureAdvance` **0.04**	SET_PRESSURE_ADVANCE ADVANCE=# (Klipper only)
`Dwell`	`Dwell` **time S**|**ms** (optional unit)	`Dwell` **0.5 S** or `Dwell` **500 ms**	G4 P# (P is milliseconds). If unit omitted, defaults to seconds. Example: `Dwell 0.5` → G4 P500

Advanced Features & Concepts

1. Parametric Geometry Engine

Bellerophon goes beyond simple macro scripts by acting as a **Parametric DSL**. Because the compiler evaluates mathematical expressions at build-time, you can create complex, procedural geometry that scales based on variables or mathematical functions.

- **Global Math:** Use `sin`, `cos`, `tan`, `sqrt`, `abs`, `sign`, `^`, and `pi` anywhere an expression is expected (e.g., coordinates, temperatures, or speed).
- **Procedural Motion:** By combining `Brepeat` loops with trigonometric functions, you can generate spirals, waves, and curves that are impossible to maintain by hand.

2. The "Override" System (Auto vs. Manual)

Bellerophon employs an intelligent extrusion engine designed to simplify your workflow while retaining full manual control:

> **Note:**
>
> **Execution Priority:**
>
> 1. **Manual Override:** If an `e=...` value is provided in a `MoveTo` command, it is **always** prioritized.
> 2. **Automatic Calculation:** If `EnableAutoExtrude = 1` is active and no manual `e` is provided, the compiler calculates the required filament volume based on your defined nozzle, filament diameter, and layer height.
> 3. **Travel Move:** If `EnableAutoExtrude = 0` and no `e` is provided, the printer performs a pure travel move (no extrusion).

3. Procedural Layering (`Layer`)

The `Layer` command automates vertical iteration. It not only repeats your macro block, but it also automatically injects a Z-hop (G91 move) between layers using your defined `LayerHeight`.

```
Layer 50
    MoveTo x=i*2 y=50
end
```

This is the engine behind "vase mode" style procedural printing, allowing you to build up volume automatically without manually managing Z-heights.

4. Variable Scoping

Variables are defined via `name = expression`. They are **local to the macro**. This prevents variable name collisions across different macros, ensuring your code remains modular and safe for inclusion in larger printer config files.

Loops & Arithmetic Operators

Bellerophon supports two types of loops: `repeat` for static repetition and `Brepeat` for dynamic loops.

> **Note:**
>
> **Positioning Context in Loops:** The behavior of movement inside a loop depends heavily on the active mode:
>
> - **Relative (G91):** Coordinates are calculated as offsets from the last position. Each move builds on the previous one. Best for iteratively forming shapes, spirals, zig-zags, or any flowing patterns.
> - **Absolute (G90):** Coordinates are calculated from a fixed origin on the bed. Each move goes to a specific point. Use this only when you need precise positions that don't depend on the previous move, like moving to a center point or fixed markers.

Example: Absolute vs Relative in Loops

The same loop code produces very different results depending on the active mode:

```
# Absolute Mode Example
Absolute
MoveTo x=10
Brepeat 4
    MoveTo x+=5
end
```

```
# Relative Mode Example
Relative
MoveTo x=10
Brepeat 4
    MoveTo x+=5
end
```

Output:

```
# Absolute Mode Output
[gcode_macro Absolute!-tester]
gcode:
```

```
    G90
    G1 X10.000
    G1 X15.000
    G1 X20.000
    G1 X25.000
    G1 X30.000


# Relative Mode Output
[gcode_macro Relative!-tester]
gcode:
    G91
    G1 X10.000
    G1 X5.000
    G1 X5.000
    G1 X5.000
    G1 X5.000
```

> **Note:**
>
> **Key observation:** In absolute mode, `x+=5` calculates a new absolute position (10+5=15, then 15+5=20, etc.). In relative mode, it adds 5 to the current position each time, creating equal-sized steps from wherever the previous move ended.

Assignment Operators

Operators allow for precise control during movement:

- `=` Direct Assignment
- `+=` Add to current
- `-=` Subtract from current
- `*=` Multiply current
- `/=` Divide current

1. Static Repeat (`repeat`)

The `repeat` command simply duplicates the enclosed G-code commands a specified number of times during compilation.

```
repeat 3
    MoveTo x+=10 y+=10
    Respond MSG "Moving..."
end
```

Output (repeat):

```
G1 X10.0 Y10.0
RESPOND MSG="Moving..."
G1 X20.0 Y20.0
RESPOND MSG="Moving..."
G1 X30.0 Y30.0
RESPOND MSG="Moving..."
```

> **Note:**
>
> The commands are physically duplicated in the output G-code. Great for simple, fixed repetitions.

2. Dynamic Loop (`Brepeat`)

The **`Brepeat`** command creates a loop that supports arithmetic expressions and mathematical functions. Unlike **`repeat`**, this loop exposes an iterator variable `i` which increments each iteration starting from 0. Expressions are evaluated during compilation, producing numeric G-code values.

> **Note:**
>
> **Important:** The iterator `i` is only valid inside **`Brepeat`** loops, but **mathematical functions (sin, cos, tan, sqrt, abs, sign) and the power operator (^) are globally available anywhere an expression is allowed** (e.g., in `MoveTo`, `Heat`, `SetSpeed`, etc.).

Example: Using the iterator

```
Brepeat 5
    MoveTo x=i
```

```
   end
```

Example: Mathematical Expressions (global availability)

Bellerophon supports arithmetic expressions anywhere: including exponentiation `^`, absolute value `abs()`, signum `sign()`, and the constant `pi`.

```
   Brepeat 3
       MoveTo x=i^2 * 5 + abs(i-2)
   end
   Heat extruder=180 + 20*sin(3.14159/2)
# works outside loops too
```

Example: Drawing a Square (Relative)

Using **Relative** mode ensures each move is from the previous point, naturally forming shapes when iterating.

```
   Relative
   Brepeat 4
       MoveTo x=10 y=0
       MoveTo x=0 y=10
       MoveTo x=-10 y=0
       MoveTo x=0 y=-10
   end
```

Example: Drawing a Circle (Relative)

Using **Relative** mode allows the circle to be built incrementally, keeping each move relative to the last point for smooth motion.

```
   Relative
   Brepeat 60
       MoveTo x=cos(i*0.1)*5 y=sin(i*0.1)*5
   end
```

Example: Drawing a Spiral (Relative)

Using **Relative** mode allows complex parametric shapes to be built incrementally. This spiral uses the loop counter `i` to calculate both angle and radius, creating an expanding spiral pattern.

```
# Spiral test macro

M.title "spiral_test"

MoveTo x = 160 y = 160 z= 10

Relative

SetSpeed = 6500

Brepeat 65

    MoveTo x=cos(i*10)*(i*0.1) y=sin(i*10)*(i*0.1)

end

Dwell 2 ms

Absolute

MoveTo x = 150 y = 150

Dwell 2 s

Home

M.end
```

> **Note:**
>
> **How the spiral formula works:** The expression `cos(i*10)*(i*0.1)` and `sin(i*10)*(i*0.1)` create an expanding spiral where:
>
> - `i*10` controls the angular step (angle increases quickly, making many turns)
> - `i*0.1` controls the radius growth (spiral expands outward slowly)
> - The product creates a spiral that grows in radius as it rotates
> - Using **Relative** mode means each move is an offset from the previous position

---

Example Program

```
M.title "Bed Leveling Routine"

    Absolute

    SetSpeed = 3000

    Home -> x=0 y=0

    MoveTo z=5

    Heat bed=60

    Respond MSG "Bed heating complete"

    Level

    LoadBedMesh "default"

    MoveTo x=100 y=100

    ProbeCalibrate

    SetPressureAdvance 0.04

    ResetExtruder

    RelativeExtrusion

    Move right

    Move up

    Pause

    Resume

    Cooldown

M.end
```